

Reporte de Hallazgos de Code Smell.

Evaluación de Calidad de Código



REPORTE DE HALLAZGOS.

Este documento presenta los hallazgos de "code smell" detectados en el código fuente del proyecto. Estos hallazgos reflejan aspectos del código que, aunque pueden no generar errores inmediatos, pueden llevar a problemas de mantenimiento, legibilidad y eficiencia a largo plazo. Además, se sugieren mejoras y mejores prácticas para garantizar un código más limpio, eficiente y fácil de mantener.

OBJETIVO DEL REPORTE.

El objetivo principal de este reporte es identificar áreas problemáticas en el código para mejorar la calidad general del mismo. El término "code smell" hace referencia a patrones de diseño y escritura de código que, aunque no son errores explícitos, indican una posible deficiencia en la estructura del software que puede dificultar la comprensión, el mantenimiento o la expansión del mismo.

HALLAZGOS.

A continuación se presentan los hallazgos más importantes en el código, junto con una breve descripción, el impacto potencial de cada uno y las recomendaciones para corregirlo.

Código Smell	Descripción	Impacto	Recomendación
Duplicación de Código	La función <code>calcular_media</code> y otras funciones similares tienen lógica repetida, lo que puede llevar a errores si se requieren modificaciones en múltiples lugares.	Mantenimiento difícil: La duplicación de lógica puede resultar en código más propenso a errores, especialmente cuando se realizan modificaciones o se agregan nuevas características.	Refactorizar el código: Se recomienda crear funciones genéricas que manejen operaciones estadísticas comunes, como sumas y promedios, y reutilizarlas en lugar de repetir el mismo código.
Funciones Grandes	La función principal es excesivamente larga y realiza varias tareas diferentes, lo que la hace difícil de	Baja legibilidad: El código se vuelve difícil de entender cuando una sola función realiza múltiples tareas sin	Dividir en funciones más pequeñas: Separar la función principal en funciones más pequeñas y específicas, cada una encargada de una tarea particular. Esto mejorará la

Reporte de Hallazgos de Code Smell.

Evaluación de Calidad de Código

	leer, entender y mantener.	un enfoque claro. Esto puede dificultar el mantenimiento y la detección de errores.	legibilidad y facilitará el mantenimiento.
Nombres Genéricos	Nombres como simulacion_binomial y tirar_moneda son poco específicos y no describen claramente su propósito o lo que hacen exactamente.	Confusión sobre la funcionalidad: Los nombres poco descriptivos dificultan la comprensión del propósito de las funciones, especialmente en proyectos grandes o cuando otros desarrolladores trabajan en el código.	Renombrar funciones: Es importante utilizar nombres claros y descriptivos que reflejen el comportamiento exacto de las funciones. Por ejemplo, simulacion_binomial podría llamarse realizar_simulacion_binomial o algo similar.
Uso Innecesario de Listas	En algunas funciones, como en la creación de las listas resultado, estas se utilizan innecesariamente cuando los datos pueden ser generados y procesados directamente sin almacenarlos en listas temporales.	Ineficiencia: El uso innecesario de listas adicionales aumenta el consumo de memoria y disminuye la eficiencia.	Optimizar el uso de listas: Evitar la creación de listas temporales cuando no son necesarias. En lugar de eso, procesar los datos directamente o utilizar generadores si es apropiado.
Falta de Manejo de Errores	El código no incluye validación de entradas ni manejo de excepciones en caso de que ocurran errores. Por ejemplo, en las funciones que realizan cálculos, no se verifica si los datos de entrada son válidos o si el programa puede	Riesgo de fallos en producción: La falta de manejo de errores puede resultar en excepciones no controladas que interrumpen la ejecución del programa o generan resultados incorrectos.	Implementar manejo de excepciones: Introducir bloques try-except para manejar posibles excepciones y validar las entradas antes de procesarlas. Esto ayudará a prevenir fallos en el sistema.

Reporte de Hallazgos de Code Smell.

Evaluación de Calidad de Código

	manejar casos inesperados.		
Complejidad Ciclomática	La función <code>calcular_varianza</code> tiene varios ciclos anidados, lo que aumenta la complejidad del código y hace que sea más difícil de entender y modificar.	Dificultad en mantenimiento y expansión: El código con alta complejidad ciclomática puede ser más difícil de modificar y expandir, ya que hay más caminos de ejecución que deben ser analizados.	Simplificar la lógica: Reducir la complejidad ciclomática refactorizando el código para usar funciones más simples y claras. Si es posible, evitar el uso de ciclos anidados innecesarios.

Conclusión y Recomendaciones Generales

Los hallazgos presentados reflejan varios problemas típicos en el código fuente que pueden comprometer la calidad, la eficiencia y la mantenibilidad del proyecto. Las recomendaciones propuestas buscan resolver estos problemas mediante la refactorización y la adopción de mejores prácticas de programación, lo que no solo mejorará el rendimiento del código, sino que también facilitará la colaboración en el equipo de desarrollo.

Acciones Sugeridas:

1. **Refactorización:** Adoptar principios de diseño limpio, como la eliminación de duplicación de código y la mejora de la legibilidad mediante funciones pequeñas y bien nombradas.
2. **Manejo de Errores:** Implementar un manejo adecuado de excepciones y validaciones de entradas para evitar fallos imprevistos.
3. **Optimización del Código:** Reducir el uso innecesario de estructuras de datos como listas temporales, utilizando enfoques más eficientes.

Al seguir estas recomendaciones, el código no solo mejorará en términos de calidad y eficiencia, sino que también será más fácil de mantener y ampliar en el futuro.